

Module: tf.compat.v1.nn Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/compat/v1/nn

Public API for tf._api.v2.nn namespace

Function Signatures

```
all_candidate_sampler(...)  
approx_max_k(...)
```

Documentation

Public API for tf._api.v2.nn namespace

Modules

experimental module: Public API for tf._api.v2.nn.experimental namespace

rnn_cell module

Functions

`all_candidate_sampler(...)`: Generate the set of all classes.

`approx_max_k(...)`: Returns max k values and their indices of the input operand in

an approximate manner.

`approx_min_k(...)`: Returns min k values and their indices of the input operand in an approximate manner.

`atrous_conv2d(...)`: Atrous convolution (a.k.a. convolution with holes or dilated convolution).

`atrous_conv2d_transpose(...)`: The transpose of `atrous_conv2d`.

`avg_pool(...)`: Performs the average pooling on the input.

`avg_pool1d(...)`: Performs the average pooling on the input.

`avg_pool2d(...)`: Performs the average pooling on the input.

`avg_pool3d(...)`: Performs the average pooling on the input.

`avg_pool_v2(...)`: Performs the avg pooling on the input.

`batch_norm_with_global_normalization(...)`: Batch normalization.

`batch_normalization(...)`: Batch normalization.

`bias_add(...)`: Adds bias to value.

`bidirectional_dynamic_rnn(...)`: Creates a dynamic version of bidirectional recurrent neural network. (deprecated)

`collapse_repeated(...)`: Merge repeated labels into single labels.

`compute_accidental_hits(...)`: Compute the position ids in `sampled_candidates` matching `true_classes`.

`compute_average_loss(...)`: Scales per-example losses with `sample_weights` and computes their average.

`conv1d(...)`: Computes a 1-D convolution of input with rank ≥ 3 and a 3-D filter. (deprecated argument values) (deprecated argument values)

`conv1d_transpose(...)`: The transpose of `conv1d`.

`conv2d(...)`: Computes a 2-D convolution given 4-D input and filter tensors.

`conv2d_backprop_filter(...)`: Computes the gradients of convolution with respect to the filter.

`conv2d_backprop_input(...)`: Computes the gradients of convolution with respect to the input.

`conv2d_transpose(...)`: The transpose of `conv2d`.

`conv3d(...)`: Computes a 3-D convolution given 5-D input and filter tensors.

`conv3d_backprop_filter(...)`: Computes the gradients of 3-D convolution with respect to the filter.

`conv3d_backprop_filter_v2(...)`: Computes the gradients of 3-D convolution with respect to the filter.

`conv3d_transpose(...)`: The transpose of `conv3d`.

`conv_transpose(...)`: The transpose of convolution.

`convolution(...)`: Computes sums of N-D convolutions (actually cross-correlation).

`crelu(...)`: Computes Concatenated ReLU.

`ctc_beam_search_decoder(...)`: Performs beam search decoding on the logits given in input.

`ctc_beam_search_decoder_v2(...)`: Performs beam search decoding on the logits given in input.

`ctc_greedy_decoder(...)`: Performs greedy decoding on the logits given in input (best path).

`ctc_loss(...)`: Computes the CTC (Connectionist Temporal Classification) Loss.

`ctc_loss_v2(...)`: Computes CTC (Connectionist Temporal Classification) loss.

`ctc_unique_labels(...)`: Get unique labels and indices for batched labels for `tf.nn.ctc_loss`.

`depth_to_space(...)`: `DepthToSpace` for tensors of type T.

`depthwise_conv2d(...)`: Depthwise 2-D convolution.

`depthwise_conv2d_backprop_filter(...)`: Computes the gradients of depthwise convolution with respect to the filter.

`depthwise_conv2d_backprop_input(...)`: Computes the gradients of depthwise convolution with respect to the input.

`depthwise_conv2d_native(...)`: Computes a 2-D depthwise convolution.

`depthwise_conv2d_native_backprop_filter(...)`: Computes the gradients of depthwise convolution with respect to the filter.

`depthwise_conv2d_native_backprop_input(...)`: Computes the gradients of depthwise convolution with respect to the input.

`dilation2d(...)`: Computes the grayscale dilation of 4-D input and 3-D filter tensors.

`dropout(...)`: Computes dropout. (deprecated arguments)

`dynamic_rnn(...)`: Creates a recurrent neural network specified by `RNNCell` cell. (deprecated)

`elu(...)`: Computes the exponential linear function.

`embedding_lookup(...)`: Looks up embeddings for the given ids from a list of tensors.

`embedding_lookup_sparse(...)`: Looks up embeddings for the given ids and weights from a list of tensors.

`erosion2d(...)`: Computes the grayscale erosion of 4-D value and 3-D kernel tensors.

`fixed_unigram_candidate_sampler(...)`: Samples a set of classes using the provided (fixed) base distribution.

`fractional_avg_pool(...)`: Performs fractional average pooling on the input. (deprecated)

`fractional_max_pool(...)`: Performs fractional max pooling on the input. (deprecated)

`fused_batch_norm(...)`: Batch normalization.

`in_top_k(...)`: Says whether the targets are in the top K predictions.

`l2_loss(...)`: L2 Loss.

`l2_normalize(...)`: Normalizes along dimension axis using an L2 norm. (deprecated arguments)

`leaky_relu(...)`: Compute the Leaky ReLU activation function.

`learned_unigram_candidate_sampler(...)`: Samples a set of classes from a distribution learned during training.

`local_response_normalization(...)`: Local Response Normalization.

`log_poisson_loss(...)`: Computes log Poisson loss given `log_input`.

`log_softmax(...)`: Computes log softmax activations. (deprecated arguments)

`log_uniform_candidate_sampler(...)`: Samples a set of classes using a log-uniform (Zipfian) base distribution.

`lrn(...)`: Local Response Normalization.

`max_pool(...)`: Performs the max pooling on the input.

`max_pool1d(...)`: Performs the max pooling on the input.

`max_pool2d(...)`: Performs max pooling on 2D spatial data such as images.

`max_pool3d(...)`: Performs the max pooling on the input.

`max_pool_v2(...)`: Performs max pooling on the input.

`max_pool_with_argmax(...)`: Performs max pooling on the input and outputs both max values and indices.

`moments(...)`: Calculate the mean and variance of x.

`nce_loss(...)`: Computes and returns the noise-contrastive estimation training loss.

`normalize_moments(...)`: Calculate the mean and variance of based on the sufficient statistics.

`pool(...)`: Performs an N-D pooling operation.

`quantized_avg_pool(...)`: Produces the average pool of the input tensor for quantized